Introduction to Algorithmic Differentiation

Everything you ever wanted to know but were afraid to ask

Jean-Luc Bouchot (INRIA Saclay)
Sri Hari Krishna Narayanan (Argonne National Laboratory)

June 23, 2026

Outline

What can you expect from this tutorial?

Participants will:

- Understand the basics of algorithmic differentiation (AD) with respect to other approaches (finite differences, symbolic differentiation)
- Learn how to use AD tools for differentiating
- See and implement examples of forward and reverse mode AD
- Discuss common pitfalls and best practices for self diagnosis
- ... maybe even want to get involved in R & D of AD tools!

What we expect from you?

This is a diverse crowd at JDEV. We assume some basic understanding of the following, although we try to review the most important background material:

- Basic calculus (chain rule, partial derivatives)
- Basic programming (variables, loops, functions)
- Some familiarity with Fortran/C (but we will try to keep it as language-agnostic as possible)
- Some familiarity of numerical methods
- Some familiarity with optimization and machine learning (but not required)

How the session will unfold

We will have a mix of theory and practice. Here is a tentative schedule

- Introduction and success stories
- Theory: Basics of AD
- Some hands-on work with `Tapenade` in forward mode
- Debugging and validation via `context` mode
- Analysis of compute times and `vector` mode
- Introduction of `reverse` mode
- Discussion / questions

Most of the codes are available in `C` and `Fortran` and, to some extent `Jax/Python`.

How to evaluate differentiated code?

Roughly two main paradigms:

- source transformation: Parse, analyse, and generate a source code implementing the derivatives
- or overloading: Update the source code to adapt active `float`, `real` to `aDouble`, and link with arithmetic library on `aDouble`'s.

Pros and cons:

	O-O	ST-AD
Dev	easy(-ier)	Demanding
Code optim.	None	Various analysis, simplifications
Efficiency	Complete unroling	Loops
Adaptability	Frictionless for exotic constructs and controls	Demanding

Abstractions

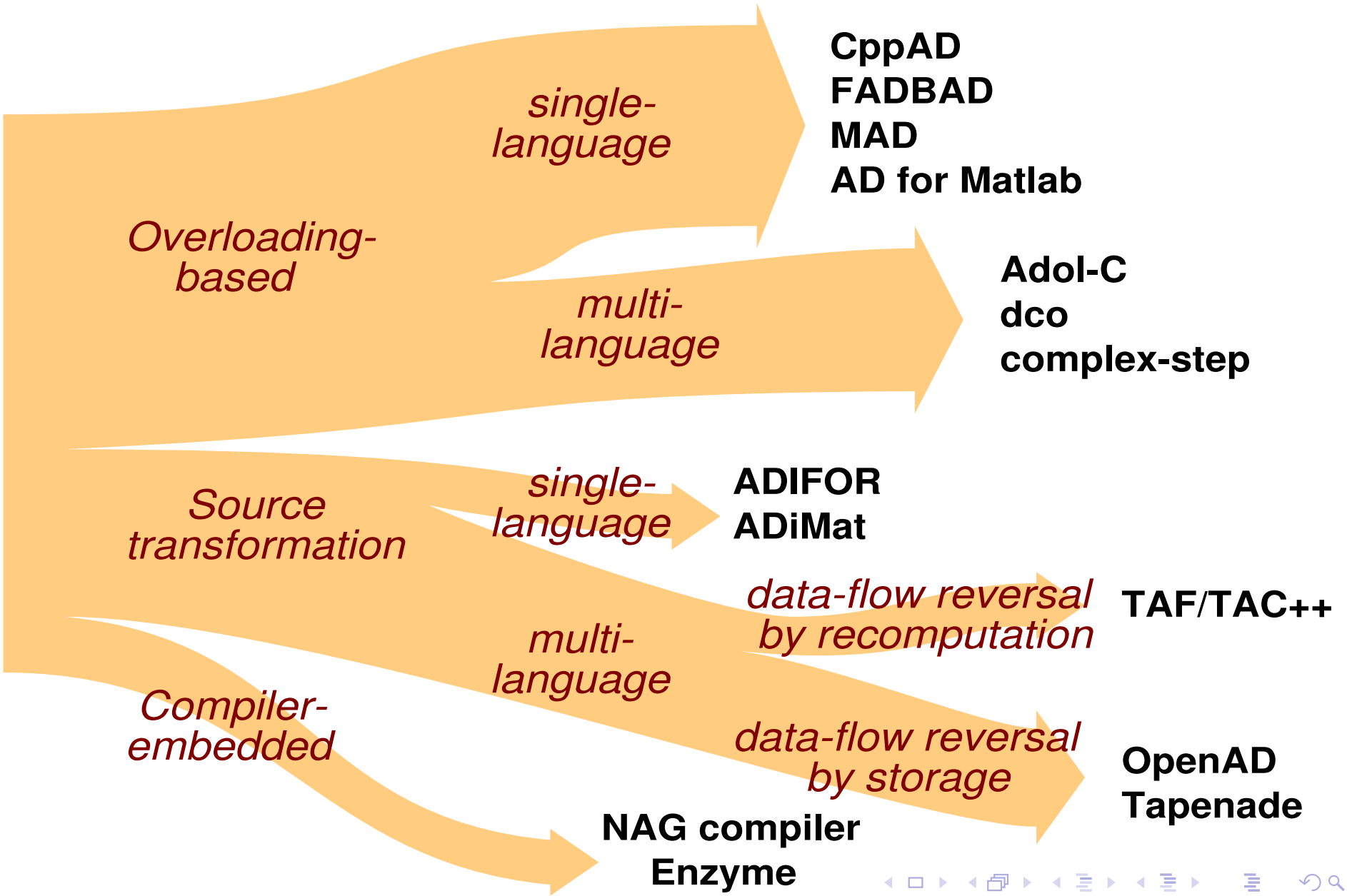
What code should be differentiated? Two main ideas:

- High level code (`ForTran`, `C++`, `Julia`...) and generate high level code
- Differentiate some *precompiled code* (`LLVM`) and geneate mid level-code

Pros and cons:

	High	Mid
Versatility	Low	High
Code optim	Static+ compil.	Twice
Readability	High	Low
Hand tuned	Easy	?

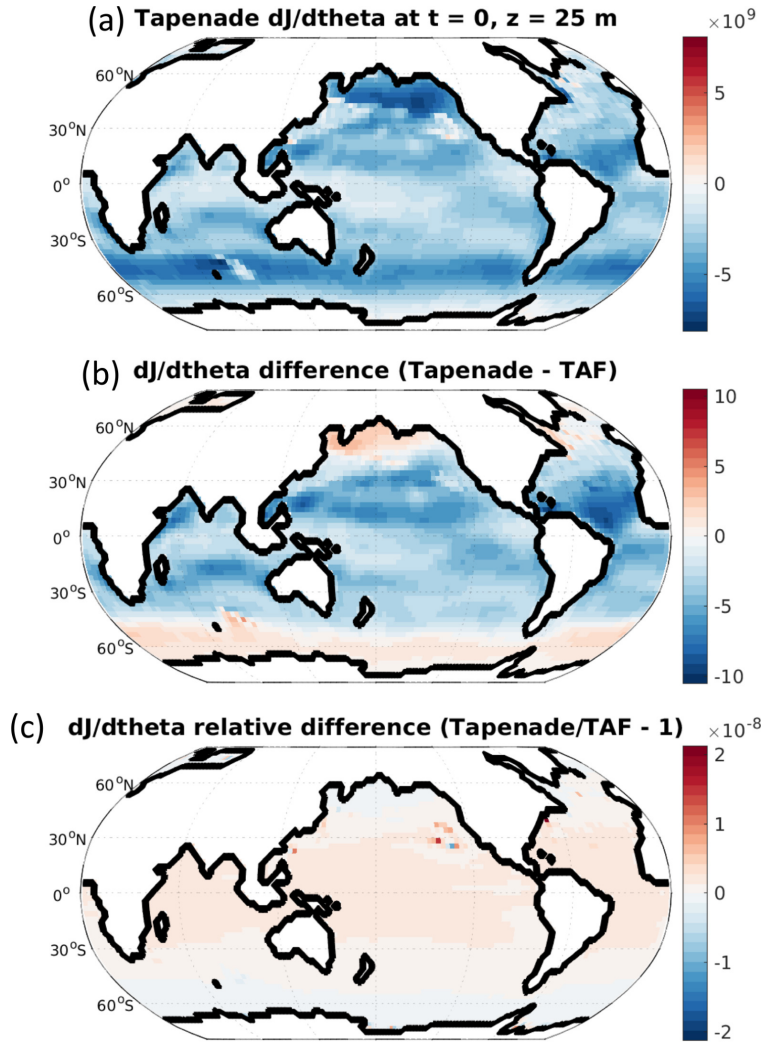
A taxonomy of AD tools



What are some successful uses of AD?

MITgcm

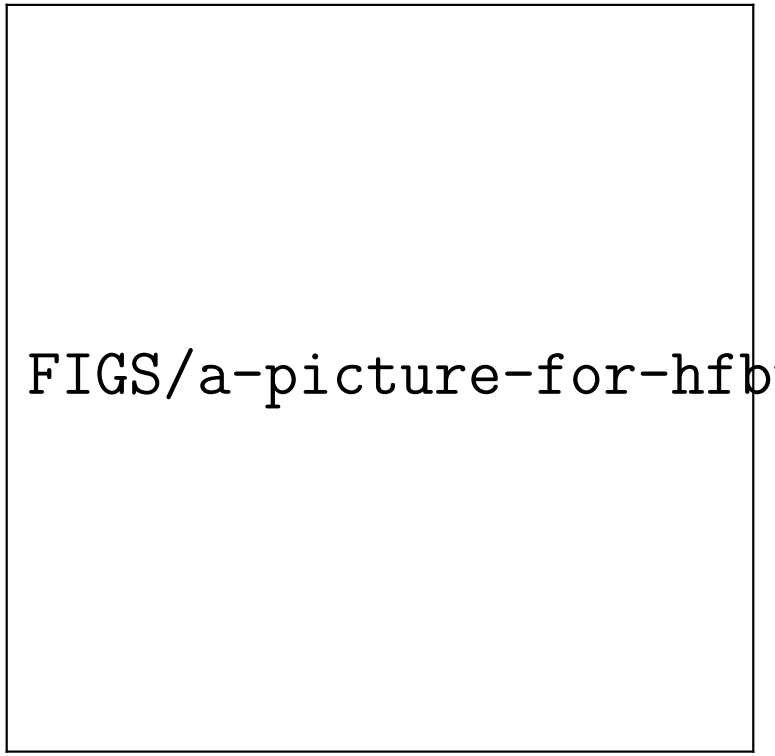
Compute XXX



What are some successful uses of AD?

HBTHO

Compute XXX



Why Do We Need Derivatives?

Common tasks:

- Optimization loop
- Sensitivity analysis
- Data assimilation
- Machine learning
- PDE-constrained problems

Key question:

How do we compute derivatives of *programs* accurately and **efficiently**?

Three Approaches (1/3)

Finite Differences

- Easy
- Numerical error
- Expensive in high dimension
- Parametrization

Example

Let $f : \mathbb{R}^p \to \mathbb{R}$. Each derivative implies the computation of

$$\frac{\partial f}{\partial \mathbf{x}_i} \approx \frac{f(\mathbf{x} + \varepsilon \mathbf{e}_i) - f(\mathbf{x}[-\varepsilon \mathbf{e}_i])}{[2]\varepsilon}$$

Computing the gradient costs $[2\times]p$ the cost of a single function evaluation.

Three Approaches (2/3)

Symbolic Differentiation

- Exact
- Expression swell
- Hard for real codes

Example

Consider the function $f : \mathbb{R} \to \mathbb{R}$ defined as $f(x) = x^2 \sin(x^2/2 + 3x)$. Its symbolic derivative is given by

$$\frac{\mathrm{d}f}{\mathrm{d}x}(x) = 2x \sin(x^2/2 + 3x) + x^2(x + 3) \cos(x^2/2 + 3x)$$

Three Approaches (3/3)

Algorithmic Differentiation

- Exact up to machine precision
- Works on programs
- Efficient

- AD differentiates code, not formula!
- AD factorizes / reuses variables

Forward diff by dual numbers

Dual numbers

Let every number embed its differentiated value. Define:

$$\mathbf{x} = x + \dot{x}\varepsilon$$

with the fictitious number $\varepsilon^2 = 0$.

ε is not *some small number*, but redefines the arithmetic on real numbers.

Dual number propagation

We propagate derivatives together with values.

$$\mathbf{x} \rightarrow (x, \dot{x})$$

Each operation updates both.

Dual number propagation

We propagate derivatives together with values.

$$\mathbf{x} \to (x, \dot{x})$$

Each operation updates both.

Example

Consider the univariate function $f(x) = x^2$.

$$\mathbf{x}^2 = (x + \dot{x}\varepsilon)^2 = x^2 + 2x\dot{x}\varepsilon \left[+\dot{x}^2\varepsilon^2\right].$$

Key insight

Forward AD is ordinary arithmetic on dual numbers.

Forward Mode Example

Given the function $z = f(x) = \sin(x^2)$
Propagation table:

Variable	Value	Derivative
x	x	$\dot{x}$
$y = x^2$	x^2	$2x\dot{x}$
$z = \sin(y)$	$\sin(y)$	$\cos(y) \cdot 2x\dot{x}$

Let's try

- $f(x) = \sin(x)$
- $f(x) = \cos(x)$
- $f(x) = \sqrt{x}$

Let's try

sin

$$\begin{aligned}\sin(\mathbf{x}) &= \sin(x)\cos(\dot{x}\varepsilon) + \cos(x)\sin(\dot{x}\varepsilon)\\ &= \sin(x)\left(1 + \mathcal{O}(\dot{x}^2\varepsilon^2)\right) + \cos(x)(\dot{x}\varepsilon + \mathcal{O}(\dot{x}^3\varepsilon^3))\\ &= \sin(x) + \cos(x)\dot{x}\varepsilon\end{aligned}$$

cos

$$\begin{aligned}\cos(\mathbf{x}) &= \cos(x)\cos(\dot{x}\varepsilon) - \sin(x)\sin(\dot{x}\varepsilon)\\ &= \cos(x)\left(1 + \mathcal{O}(\dot{x}^2\varepsilon^2)\right) - \sin(x)(\dot{x}\varepsilon + \mathcal{O}(\dot{x}^3\varepsilon^3))\\ &= \cos(x) - \sin(x)\dot{x}\varepsilon\end{aligned}$$

$\sqrt{\cdot}$

$$\sqrt{\mathbf{x}} = \sqrt{x}\sqrt{1 + \frac{\dot{x}}{x}\varepsilon} \qquad = \sqrt{x}\left(1 + \frac{1}{2}\frac{\dot{x}}{x}\varepsilon + \mathcal{O}(\dot{x}^2\varepsilon^2)\right)$$

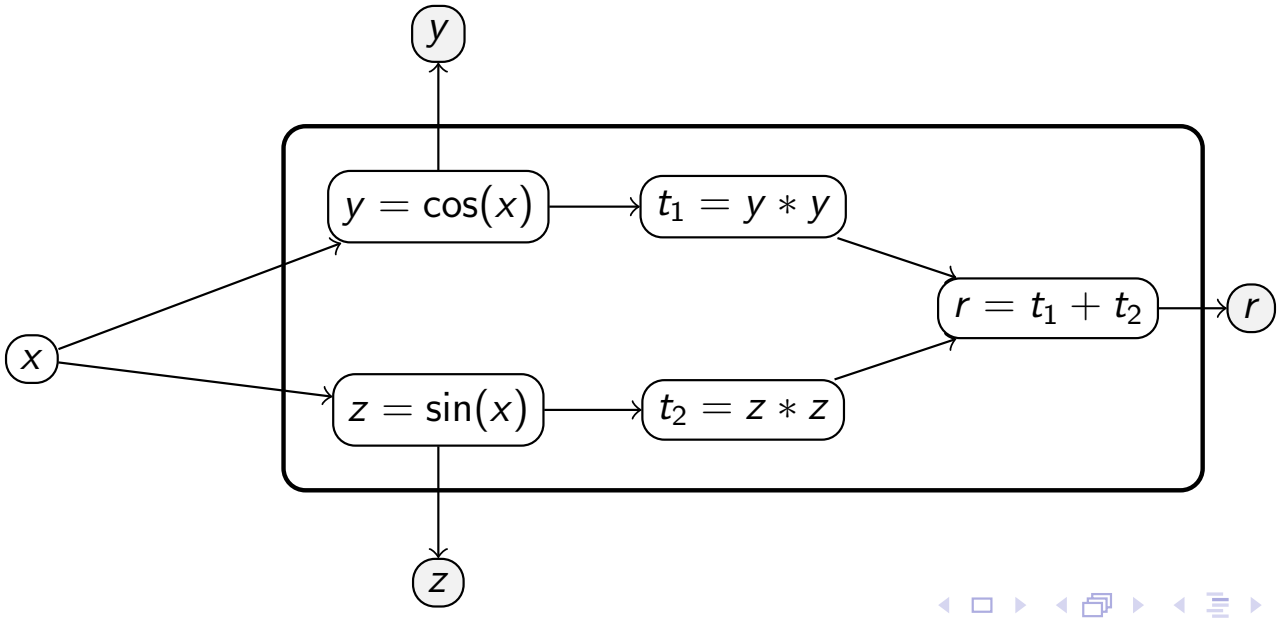
Differential examples

Differenting elementary operations:

Operations	Primal	Derivative
`add`	$z = x + y$	$\mathrm{d}z = \mathrm{d}x + \mathrm{d}y$
`mul`	$z = x \times y$	$\mathrm{d}z = y\mathrm{d}x + x\mathrm{d}y$
`sin`	$z = \sin(x)$	$\mathrm{d}z = \cos(x)\mathrm{d}x$

Programs as Computational Graphs

```
  INPUT: x
  OUTPUT: y,z,r
y = cos(x)
z = sin(x)
t1 = y*y
t2 = z*z
r = t1+t2
```



Forward AD via Dual Numbers

Seed the input perturbation $\mathrm{d}x/\mathrm{d}x = 1$ in $\mathbf{x} = x + 1 \cdot \varepsilon, \quad \varepsilon^2 = 0.$
Propagate through the program:

$$\begin{aligned}
\mathbf{y} &= \cos(\mathbf{x}) = \cos(x) - \sin(x)\varepsilon, & \dot{y} &= -\sin(x), \\
\mathbf{z} &= \sin(\mathbf{x}) = \sin(x) + \cos(x)\varepsilon, & \dot{z} &= \cos(x), \\
\mathbf{t}_1 &= \mathbf{y}^2 = y^2 + 2y\dot{y}\varepsilon, & \dot{t_1} &= 2y\dot{y}, \\
\mathbf{t}_2 &= \mathbf{z}^2 = z^2 + 2z\dot{z}\varepsilon, & \dot{t_2} &= 2z\dot{z}, \\
\mathbf{r} &= \mathbf{t}_1 + \mathbf{t}_2 = r + \left(\dot{t_1} + \dot{t_2}\right)\varepsilon & \dot{z} &= \cos(x), \dot{r} = \dot{t_1} + \dot{t_2}.
\end{aligned}$$

The initialization are called seeds.

This is AD

```
SUBROUTINE FOO(x,    y,    z,    r,   )

  REAL x,y,z,r

  y = COS(x)

  z = SIN(x)

  r = y*y + z*z
END
```

This is AD

```
SUBROUTINE FOO(x,xd,y,yd,z,zd,r,rd)
  REAL yd,zd,rd,xd
  REAL x,y,z,r
  yd = -SIN(x)*xd
  y = COS(x)
  zd = COS(x)*xd
  z = SIN(x)
  rd = 2*y*yd + 2*z*zd
  r = y*y + z*z
END
```

Forward Mode review

Forward / tangent mode idea

Forward AD propagates perturbations: "How much is this output influenced by a small change in this input?"

Source transformation AD

Systematic application of the chain rule on every instruction in the code, interweaving tangent and primal code

Let's get started with Tapenade

Various ways to test Tapenade:

- Online via Tapenade's servlet
- Local install
- In a docker environment
- Go to XXXX TODO: Add Tiny URL once public
- Follow the README for install / usage instructions

First example

- Go through the code `01BasicTest` and write down the actual (maths) computation
- What do you expect the differentiated code should be?
- Run Tapenade from the command line *as is* and compare with your expectations.

First example

- Go through the code `01BasicTest` and write down the actual (maths) computation
- What do you expect the differentiated code should be?
- Run Tapenade from the command line *as is* and compare with your expectations.
- The log tells you it has no root ⇒ use `-head`
- You see more variable than what you thought ⇒ by default, every differentiable *thing* is being differentiated. Be specific in the `-head quad_fun(quad_fun/x)`:

Note that you can tell Tapenade to output the results (and log files) to an external destination

```
mkdir RESULTS
tapenade -O RESULTS
```

Take-home message 1

Useful arguments for Tapenade:

- `-O` redirects the output to a directory (needs to exist)
- `-forward` or `-tangent` or `-d` (i.e. *dot*) or `-tl` forces tapenade to work out the forward mode (which is the default)
- `-head` tells tapenade which routine you want to differentiate
- `-head ROUTINE(o1,o2)/(i1,i2,i3)` allows to compute only the necessary derivatives
- The *return variable* of the function and the function itself share the same name.

Second example

Let's look at `02MultipleOutputs`. Its main function reads

```
double get_radius(double in, double* outcos, double* outsin){
  double tmp = in*in/2.0+in+1.0;

  *outcos = cos(tmp);
  *outsin = sin(tmp);

  return (*outcos)*(*outcos)+(*outsin)*(*outsin);
}
```

Run tapenade to differentiate this differentiation *head*'s output as well as the `cos` and `sin` variables with respect to the input `in`

Take-home message 2

- Differentiation tools differentiate instructions, not logic, the return value should always be 0. You may see the following piece of code:

```
return 2*(*outcos)*(*outcosd) + 2*(*outsin)*(*outsind)
```

- See how the return value is passed as an argument, while its derivative is given as the output of the diff'ed function:

```
double get_radius_d(double in, double ind, double *outcos
    double *outsin, double *outsind, double *get_radius) {
```

- Note the automatic naming conventions: `vard` for the (forward) differentiated variables and `functionName_d` for the (forward) differentiated functions.

Tangent, forward AD and directional derivatives

Jacobian Vector products

Let $f : \mathbb{R}^p \to \mathbb{R}^d$ be a function implemented by program P. The Jacobian vector product (JVP) is a function mapping an input vector $\mathbf{v}$ to the derivative of the function f along the direction $\mathbf{v}$ i.e.

$$JVP : \mathbb{R}^p \times \mathbb{R}^p \to \mathbb{R}^d, JVP_f(\mathbf{v}; \mathbf{w}) = J_f(\mathbf{w}) \cdot \mathbf{v}$$

No need for the evaluation or storage of the whole Jacobian.

Remarks

This seems overly complicated however, note:

- If $p = 1$, then usually one will have $\mathbf{w}$ represent the (scalar) data, and starting with $\mathbf{v} = 1$ will compute the whole *Jacobian*.
- Assuming $d > 1$, one needs to know in which direction the propagation is required.

Why JVPs

Newton–Krylov solvers

To solve a nonlinear system $F(\mathbf{x}) = 0$, Newton's method uses

$$J_F(\mathbf{x}_k)\,\delta_k = -F(\mathbf{x}_k), \qquad \mathbf{x}_{k+1} = \mathbf{x}_k + \delta_k.$$

In large-scale problems, the linear system is typically solved by a Krylov method (e.g. GMRES).

- For a linear system $J_F(\mathbf{x}_k)\,\delta_k = -F(\mathbf{x}_k)$, construct the Krylov subspace

$$\mathcal{K}_m = \text{span}\{r_0, J_F(\mathbf{x}_k)r_0, \ldots, J_F(\mathbf{x}_k)^{m-1}r_0\}, \quad r_0 = F(\mathbf{x}_k).$$

- Equivalently, the method only requires evaluations of the form $J_F(\mathbf{x}_k)v$.

How should I run the diff'ed code?

Question 1: I had a running main and now I have some functions. What should I do?
Question 2: How can I make sure my derivatives are correct?
$\Rightarrow$ Use `-context` mode! It

- produces a complete differentiated program, with a sample main and helps with validation / debug harnesses.
- requires a set of utility functions within `ADFirstAidKit` available via tapenade's source code: (adapt to yout path)

```
wget tapenade.inria.fr:8080/tapenade/ADFirstAidKit.tar
tar vxf ADFirstAidKit.tar
```

- allows debugging by running twice the compiled code with the environment variable `DBAD_PHASE=1` (perturbed run) and `DBAD_PHASE=2` (*natural* run).
- comparing both runs gives the divided difference baseline for validation

Running example 03

Run the same command as before, but provide the `-head` to the command line.

```
mkdir RESULTS
tapenade -O RESULT/ toBeDifferentiated.c -head "get_radius(get
```

Some care should be taken:

1. The results are *condensed*: if the differentiation involves multiple outputs, the results need to be reduced to a single number.
2. Compile things via

```
gcc -I <ADFirstAidKit> toBeDifferentiated_d.c <ADFirstAidK
```

3. Typical validation sequence:

```
DBAD_PHASE=1 ./tgt
DBAD_PHASE=2 ./tgt
```

Take-home message 3

- `-context` option allows for instrumentation of the diff'ed code.
- Order of Tapenade's command line arguments doesn't matter: all non-dashed arguments are assumed to be files
- Always debug your (differentiated) code in tangent mode.
- Run the tangent context code twice, setting the `DBAD_PHASE` variable
- Compare actual tangent computed values with the provided finite differences

Vector mode and multiple directions

What can be done if we need more than one direction?

Two options are available:

- Loop around the derivative code with the appropriate initialisation.
- Generate derivatives *in vector mode*, allowing to mutualize expressions as much as possible.

Seed matrices

Forward vector mode

Let $f : \mathbb{R}^p \to \mathbb{R}^d$ and let $S \in \mathbb{R}^{p \times r}$ be a *seed matrix*. Initialize the tangent variables by $\dot{x} = S$.
Propagating these perturbations through the program gives

$$\begin{bmatrix} | & | & & | \\ \dot{\mathbf{y}}_1 & \dot{\mathbf{y}}_2 & \dots & \dot{\mathbf{y}}_r \\ | & | & & | \end{bmatrix} \quad \dot{\mathbf{y}} = J_f(\mathbf{x})\, S \in \mathbb{R}^{q \times r}, \quad \dot{\mathbf{y}}_i = J_f(\mathbf{x}) \cdot \mathbf{s}_i.$$

Thus each column of $\dot{y}$ is the derivative of the output in one chosen direction.

- If $r = 1$, this is the usual JVP / directional derivative.
- If $S = I_p$, then $J_f(x)S = J_f(x)$ and the full Jacobian is obtained.
- If $r > 1$, several directions are computed simultaneously, reducing overhead.

Running example 04

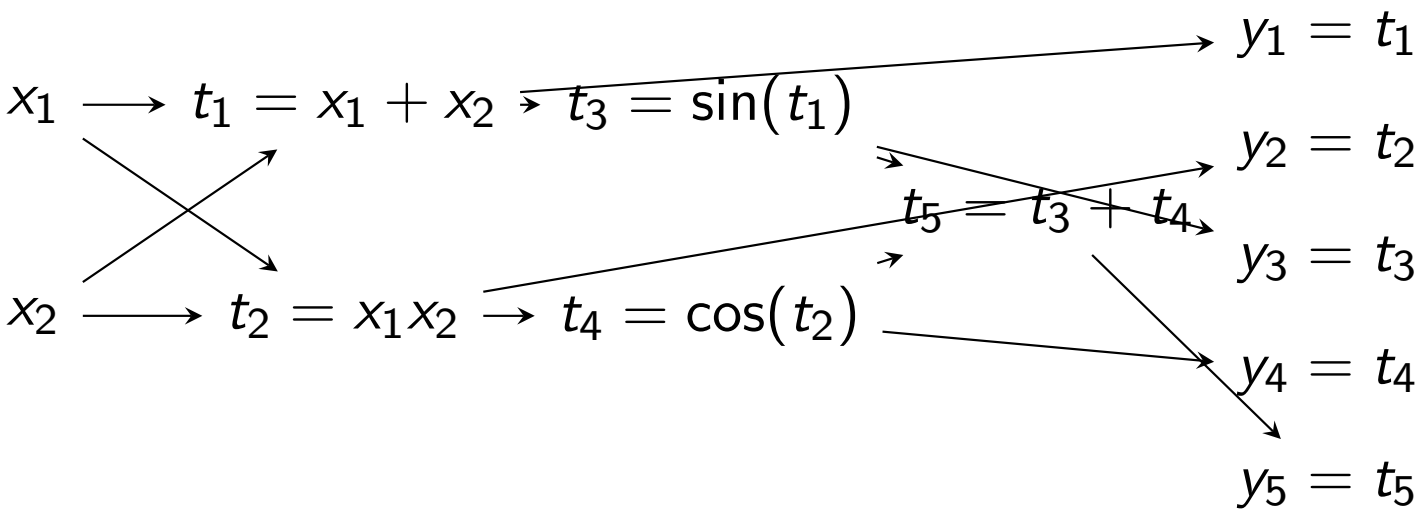
Look at example 04 and

- Run the sample code: `make; ./ctest`
- Run tapenade in forward mode, differentiating the routine `compute_matrices` in forward mode.
- Look specifically at the differentiated structures.
- Validate the derivatives using the `-context` mode and the two `DBAD_PHASE` calls.
- Re-run `Tapenade` to get ∂`aff`$/\partial$(`max_val`, `sigma`)
- Adapt the driver provided in `RESULTS` as a `driver_gradient_d.c` to compute the derivative via a `for` loop, seeding every aindependent variable one after the other.
- Compile things and run !
- Repeat the three previous operations but with the `-vector` mode for `Tapenade` (and create a `driver_gradient_dv.c`).
- Compile, run, compare.

Take-home message 4

- always make sure your code runs and always check the debug context mode
- Tapenade is able to handle many complex types (composite, vector, ...)
- *Forward Vector mode* is much faster than multiple calls to a *scalar forward mode*.
- Vector mode is called with `-vector` or `-multi`
- Need to adapt the code or include a `DIFFSIZES` file
- WARNING: The log messages are important; read them!
- `NbDirsMax` has an important role!

Toy example 1: $f : \mathbb{R}^5 \to \mathbb{R}^2$

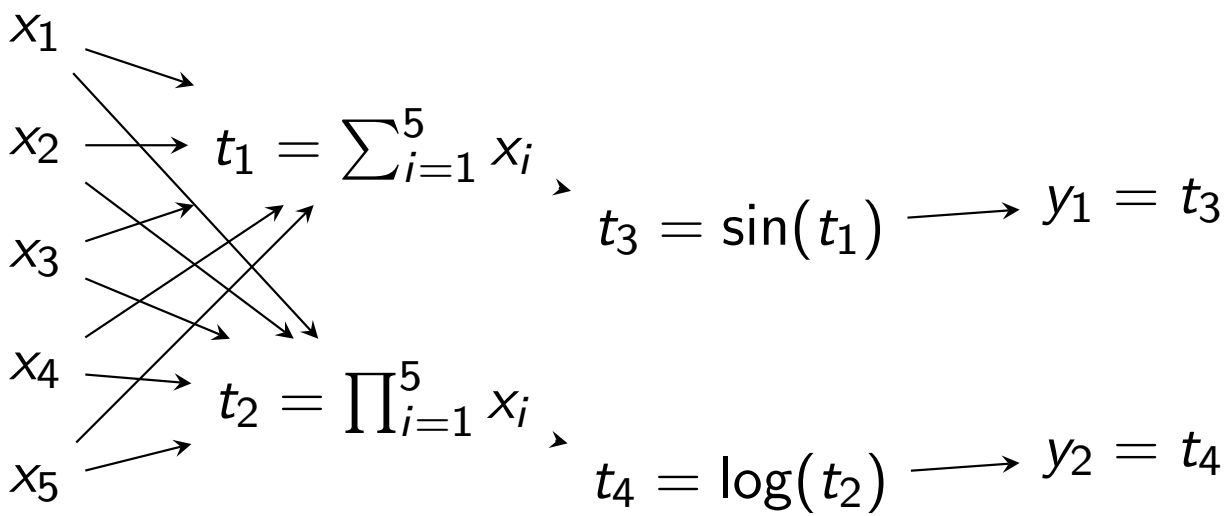


For a direction $v = (v_1, v_2)^T$, set $\dot{x}_1 = v_1$, $\dot{x}_2 = v_2$. Then

$$\begin{aligned}
\dot{y}_1 &= \dot{t}_1 = \dot{x}_1 + \dot{x}_2, \\
\dot{y}_2 &= \dot{t}_2 = x_2\dot{x}_1 + x_1\dot{x}_2, \\
\dot{y}_3 &= \dot{t}_3 = \cos(t_1)\,\dot{t}_1, \\
\dot{y}_4 &= \dot{t}_4 = -\sin(t_2)\,\dot{t}_2, \\
\dot{y}_5 &= \dot{t}_5 = \dot{t}_3 + \dot{t}_4.
\end{aligned}$$

Each equation is a JVP for one edge of the graph.

Toy example 1: $f : \mathbb{R}^2 \to \mathbb{R}^5$



For a direction $v = (v_1, \ldots, v_5)^T$, set $\dot{x}_i = v_i$. Then

$$\dot{t}_1 = \sum_{i=1}^{5} \dot{x}_i, \qquad \dot{t}_2 = t_2 \sum_{i=1}^{5} \frac{\dot{x}_i}{x_i},$$

$$\dot{y}_1 = \dot{t}_3 = \cos(t_1)\, \dot{t}_1, \qquad \dot{y}_2 = \dot{t}_4 = \frac{1}{t_2}\, \dot{t}_2.$$

Computational cost comparison

Given the previous toy examples, we want to compute the whole Jacobian or need to know the sensitivity of all inputs on every outputs.

Cost analysis 1

We need to compute two directions: $\mathbf{v} = (1, 0)^T$ and $\mathbf{w} = (0, 1)^T$ (or use the seed matrix I_2). This generates all sensitivities needed.

Cost analysis 2

We need to compute five directions: $\mathbf{s} = (1, 0, 0, 0, 0)^T$, $\mathbf{t} = (0, 1, 0, 0, 0)^T$, $\mathbf{u} = (0, 0, 1, 0, 0)^T$, $\mathbf{v} = (0, 0, 0, 1, 0)^T$, $\mathbf{w} = (0, 0, 0, 0, 1)^T$ (or use the seed matrix I_5). Doing so generates all the required sensitivities.

Left to right or right to left

A program computes **out** from **in**:

$$\mathbf{in} \to \mathrm{v}_1 \to \mathrm{v}_2 \to \cdots \to \mathrm{v}_9 \to \mathbf{out}$$

One can propagate $\frac{\mathrm{dv}}{\mathrm{d}\mathbf{in}}$ forward $\Rightarrow$ Tangent (JVPs!):

$$1.0 = \dot{\mathbf{in}} = \frac{\mathrm{d}\mathbf{in}}{\mathrm{d}\mathbf{in}} \to \frac{\mathrm{dv}_1}{\mathrm{d}\mathbf{in}} \to \frac{\mathrm{dv}_2}{d\mathbf{in}} \to \cdots \to \frac{\mathrm{dv}_9}{\mathrm{d}\mathbf{in}} \to \frac{\mathrm{d}\mathbf{out}}{\mathrm{d}\mathbf{in}}$$

One can propagate $\frac{\mathrm{d}\mathbf{out}}{\mathrm{dv}}$ backward $\Rightarrow$ Adjoint (VJPs, coming now):

$$\frac{\mathrm{d}\mathbf{out}}{\mathrm{d}\mathbf{in}} \leftarrow \frac{\mathrm{d}\mathbf{out}}{\mathrm{dv}_1} \leftarrow \frac{\mathrm{d}\mathbf{out}}{\mathrm{dv}_2} \leftarrow \cdots \leftarrow \frac{\mathrm{d}\mathbf{out}}{\mathrm{dv}_9} \leftarrow \frac{\mathrm{d}\mathbf{out}}{\mathrm{d}\mathbf{out}} = \bar{\mathbf{out}} = 1.0$$

Note that the initialization might not be just a scalar 1, but could be any seed matrix when dealing with many variables!

Costs of Tangent and Adjoint AD

f : $\mathbf{in} \in \mathbb{R}^p \rightarrow \mathbf{out} \in \mathbb{R}^d$, computed with cost P

$$\frac{\mathbf{d\ out}}{\mathbf{d\ in}} = \begin{pmatrix} * & * & * & * & * & * \\ * & * & * & * & * & * \\ * & * & * & * & * & * \\ * & * & * & * & * & * \end{pmatrix}$$

- $\frac{\mathbf{d\ out}}{\mathbf{d\ in}}$ costs $p * 4? * \mathrm{P}$ using the tangent mode
  Good if $d >= p$
- $\frac{\mathbf{d\ out}}{\mathbf{d\ in}}$ costs $d * 4? * \mathrm{P}$ using the adjoint mode
  Good if $d \ll p$ (e.g $d = 1$ for a gradient)

JVPs vs VJPs

JVPs as directional derivatives

$$JVP_f(\mathbf{w}, \mathbf{v}) = J_F(\mathbf{w}) \cdot \mathbf{v} = \lim_{h \to 0} \frac{f(\mathbf{w} + h\mathbf{v}) - f(\mathbf{w})}{h}$$

It describes the changes in the outputs given the perturbation of the inputs $\mathbf{v}$.

VJPs as differentiated projections

Consider $\mathbf{u} \in \mathbb{R}^d$ and let $g(\mathbf{w}; \mathbf{u}) = \langle u, f(\mathbf{w}) \rangle$ be the projection of the outputs of f onto the (output) direction $\mathbf{u}$. Its gradient is given by

$$\nabla_{\mathbf{w}} g(\mathbf{w}; \mathbf{u}) = \sum_{i=1}^{d} u_i \nabla_{\mathbf{w}} f_i(\mathbf{w}) = \mathbf{u}^T J_f(\mathbf{w}).$$

JVPs are precisely the projection of the perturbations in the inputs onto the objective values, while JVPs are precisely the propagation of

VJPs... reverse... adjoints

reverse

From the definition of the JVPs we can see that function values perturbations propagate to input pertubations. This is also called reverse or backward propagation (a.k.a. backprograpation).

adjoint

Output sensitivities are usually called adjoint variables and denoted with bar: $\bar{\cdot}$. The name adjoint comes from the scalar product relationship, for any $\dot{\mathbf{w}} \in \mathbb{R}^n$ and $\bar{\mathbf{w}} \in mathbbR^d$,

$$\langle J_f(\mathbf{w})\dot{\mathbf{w}}, \bar{\mathbf{w}} \rangle = \langle \dot{\mathbf{w}}, J_f(\mathbf{w})^T \bar{\mathbf{w}} \rangle$$

i.e. JVP's adjoint is VJP.

JVPs complement VJPs

A quadratic model and conjugate gradient

For a quadratic objective

$$\phi(\mathbf{x}) = \tfrac{1}{2}\mathbf{x}^T A\mathbf{x} - b^T\mathbf{x},$$

the conjugate gradient method generates iterates

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k p_k$$

where the search direction satisfies

$$p_{k+1} = r_{k+1} + \beta_k p_k, \qquad r_k = b - A\mathbf{x}_k.$$

- In a nonlinear setting, one often replaces A by the Hessian $\nabla^2\phi(\mathbf{x}_k)$.
- The Hessian-vector product needed by CG is then of the form

Adjoint Variables

Define for each variable v:

$$\bar{v} = \frac{\partial z}{\partial v}$$

Initialize:

$$\bar{z} = 1$$

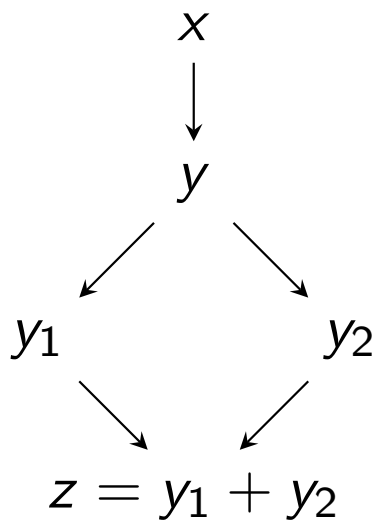
The Aha Moment: Why += ?

Consider:

```
y = x*x
z = y + y
```

Derivative:

$$\frac{\partial z}{\partial y} = \frac{\partial y_1}{\partial y} + \frac{\partial y_2}{\partial y} = 1 + 1 = 2.$$



Backward propagation:

$$\bar{y} += \bar{y}_1 = 1,$$
$$\bar{y} += \bar{y}_2 = 1.$$

Key message

Adjoints accumulate because multiple paths contribute.

General Reverse Rule

For any variable v:

$$\bar{v} = \sum_{w \in children(v)} \bar{w} \frac{\partial w}{\partial v}$$

Interpretation

Reverse mode sums contributions from all downstream paths.

Cost of Reverse Mode

Rule of thumb

Cost $\propto$ number of outputs

Best when:

- Many inputs
- Scalar (or few) output(s)

Example 05: reverse mode

This code is the same as the previous one.

- Run `Tapenade` in `-reverse` mode.
- Look at the addition of the `adStack` library (available in the `ADFirstAidKit`)
- A variable `adCount` seems to be misplaced. It is. But is has since been corrected in `Tapenade` and will be released soon.
- Some driver code has been added for you to run a comparison between forward and backward modes: look in particular at the end of the files and compare.

Take-home message 5

- Reverse mode great for one dimensional output spaces
- Some memory book-keeping required
- Stacks and memory need to be handled properly
- For advanced considerations: checkpointing is a real problem
- Reverse mode obtained via `-reverse` or `-b`
- Adoint variables are named `varb` while adjoint functions are named `func_b`. Some textttfunc_fwd and `func_bwd` also exist.

Example 06: tangent vs reverse; loop vs vector

- As should be usual with any code: familiarize yourself with it, and make sure it runs
- Run the `Tapenade` context and debug modes to make sure differentiation has a chance to succeed
- We are interested in the four dependent variables `average`, `mabs`, `std`, and `a_to_s` and the (vector of) independent variable `values`.
- The root of differentiation is the function called `compute_qoi`.
- Adapt the sample driver code in `RESULTS` into four version: forward (+ `for` loop), forward vector, reverse (+ `for` loop), and vector reverse.
- Compare the execution times.
- Don't forget to link the `ADFirstAidKit` to the compilation!

Take-home message 6

- If using loops: make sure you reinitialise the seeds
- If very few outputs, reverse is likely to be better suited
- Use vector mode whenever possible

Common Pitfalls

- Non-differentiable functions (abs, max)
- Hidden state
- Memory in reverse mode
- Aliasing in Fortran

Takeaways

- AD applies the chain rule to programs
- Forward mode: push derivatives
- Reverse mode: pull sensitivities
- Reverse requires accumulation (+=)
- Tapenade automates the process

Now let's get started!

Example Fortran Code

```
double precision function energy(x)
double precision x
energy = x*x + sin(x)
end
```

Running Tapenade

Forward mode:

```
tapenade -forward -head energy energy.f
```

Reverse mode:

```
tapenade -reverse -head energy energy.f
```

What Tapenade Generates

You will see:

- Tangent variables (forward)
- Adjoint variables (reverse)
- Two-pass structure in reverse mode

Loop Example

```
do i = 1, n
   y = y + a(i)*x(i)
end do
```

Important

AD differentiates through loops automatically.